IF2240 Basdat

Relational Model & Relational Algebra

Extended Operators

Relational Model

Overview

Relational Model Concept

- A relation is a mathematical concept based on the **ideas of sets**
- Why study this?
    - Most widely used model

Relation

- **Definition**: A relation is a named, two-dimensional table of data
- A table consists of **rows (records)** and **columns (attribute or field)**
- Requirements for a table to qualify as a relation:
    - it must have a unique name
    - every attribute value must be atomic (not multivalued nor composite)
    - every row must be unique (cant have two rows with exactly the same values for all their fields)
    - attributes (columns) in tables must have unique names
    - the order of the columns and rows may be unordered
- All relations are in 1st normal form

Attributes

- Each attribute of a relation has a name
- Domain of the attribute = the set of allowed values for each attribute
- Attribute values are required to be **atomic**
- The value null is a member of every domain
  Meanings:
    - Value **unknown**
    - Value **exists but isnt available**
    - Attribute doesnt apply to this tuple (AKA value **undefined**)
- IMPORTANT: **NULL != NULL**
    - Misalnya ada 2 row yang mengandung nilai NULL, kedua nilai NULL tsb tidak bisa disamakan.

Relation Schema & Instance

- A_1, A_2, ..., A_n are attributes

- $R = (A_1, A_2, ..., A_n)$ is a relation schema

  Example:

  instructor = (ID, name, dept_name, salary)
- The current values a relation are specified by a table
- An element t of relation r is called a tuple and is represented by a row in a table

Database Schema

- Database **schema** is the **logical structure** of the database
- Database **instance** is a **snapshot** of the **data** in the database at a given instant in time

  Example:
  - schema: instructor (ID, name, dept_name, salary)
  - Instance:

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

Why Split Information Across Relations?

Storing all information as a single relation such as

bank(account_number, balance, customer_name, ...)

results in:

- repetition of information
  - misalnya, seorang mahasiswa mengambil 6 mata kuliah, maka data mahasiswa tersebut (NIM, nama, alamat, dll.) akan terulang untuk keenam matkul tsb.
- the need for null values
  - misalnya, ada mahasiswa baru yang belum mengambil mata kuliah apapun, maka akan muncul NULL. → nilai NULL akan muncul di seluruh maba

Normalization theory deals with how to design relational schemas

Keys

Terdapat 4 key yang harus diketahui:

1. Superkey
2. Candidate keys
3. Primary key
4. Foreign key

Superkey

Let $K \subseteq R$. *K* is a superkey of *R* if values for *K* are sufficient to identify a unique tuple of each possible relation *r(R)*

customer_name	customer_street	city
Luke	Sarijadi	Bandung
Leia	Sarijadi	Bandung
Han Solo	Dago	Bandung
Rey	Menteng	Jakarta
Lando	Kuningan	Jakarta

Pada tabel di atas, hanya customer_name yang superkey karena ia menjadi pembeda antara relasi. customer_street dan city tidak bisa menjadi superkey karena ia tidak bisa menjadi pembeda (terdapat lebih dari 1 org yang tinggal di Bandung/ Jakarta/ Sarijadi Dago/ Menteng/ Kuningan).

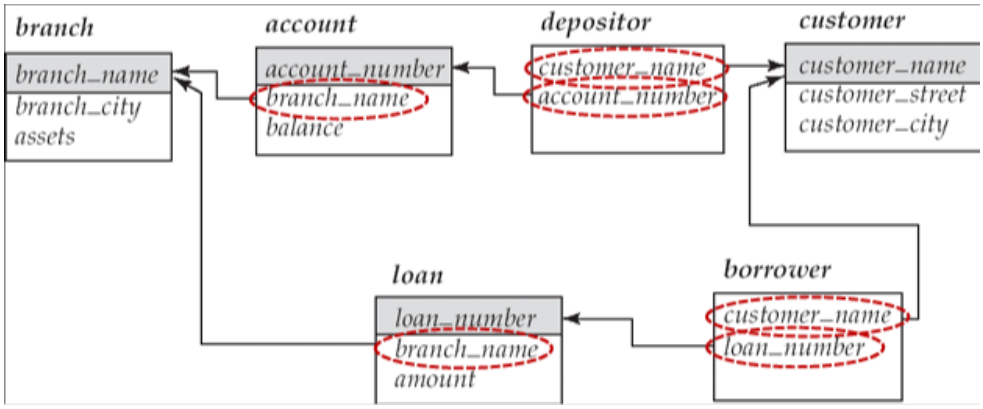
Umumnya, atribut ID digunakan sebagai superkey, karena mungkin ada 2 orang dengan nama yang sama.

Candidate and Primary Key

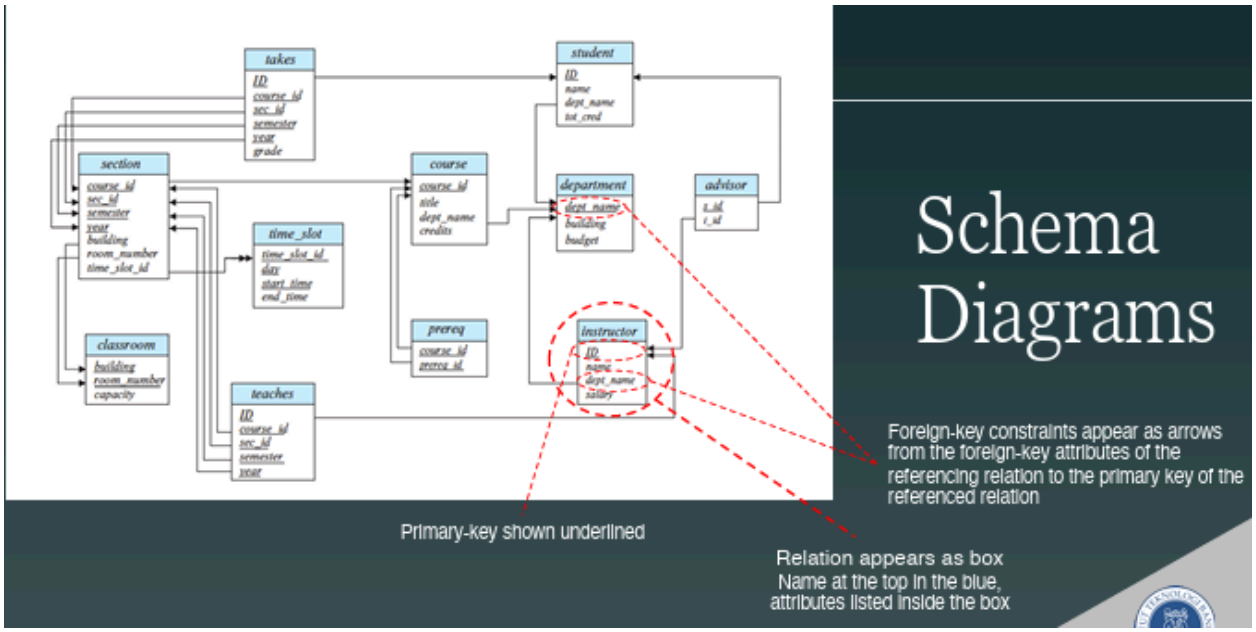
- K is a candidate key if K is minimal. For example, {customer_name} is a candidate key for customer since **it is a superkey and no subset of it is a superkey**.
- A primary key is a candidate key chosen as the principal means of identifying tuples within a relation.
  Syarat:
  - Nilainya jarang dan hampir tidak pernah berubah, contohnya nama, NIK, tanggal lahir seseorang.
- Keys can be simple (a single field) or composite (more than one field)
- Keys usually are used as indexes to speed up the response to user queries

Foreign Key

A foreign key is an attribute that corresponds to the primary key of another relation



Schema Diagram



Integrity Constraint

Domain Constraints

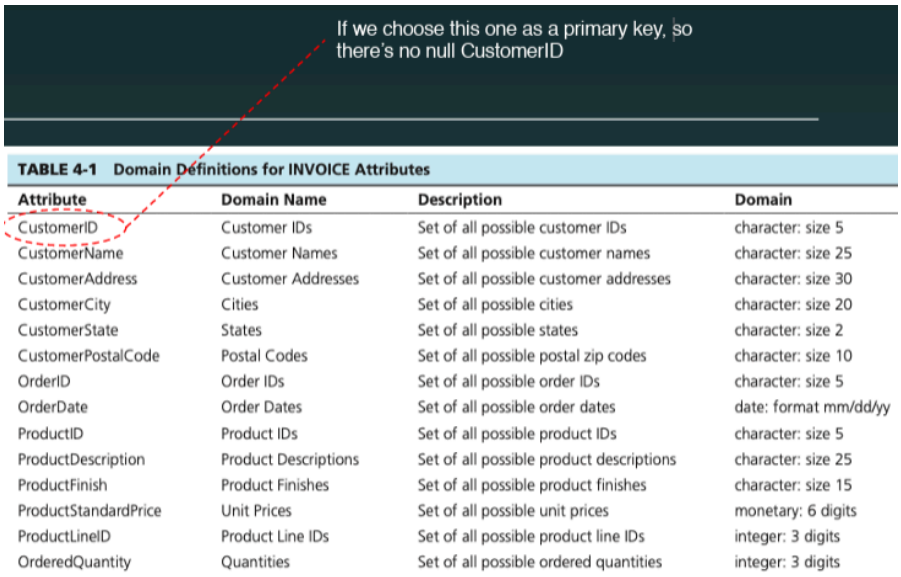
- All of the values that appear in a column of a relation must be from the same domain.
- A domain definition usually consists of the following components: domain name, meaning, data type, size or length, and allowable values or allowable range (if applicable)

TABLE 4-1 Domain Definitions for INVOICE Attributes

Attribute	Domain Name	Description	Domain
CustomerID	Customer IDs	Set of all possible customer IDs	character: size 5
CustomerName	Customer Names	Set of all possible customer names	character: size 25
CustomerAddress	Customer Addresses	Set of all possible customer addresses	character: size 30
CustomerCity	Cities	Set of all possible cities	character: size 20
CustomerState	States	Set of all possible states	character: size 2
CustomerPostalCode	Postal Codes	Set of all possible postal zip codes	character: size 10
OrderID	Order IDs	Set of all possible order IDs	character: size 5
OrderDate	Order Dates	Set of all possible order dates	date: format mm/dd/yy
ProductID	Product IDs	Set of all possible product IDs	character: size 5
ProductDescription	Product Descriptions	Set of all possible product descriptions	character: size 25
ProductFinish	Product Finishes	Set of all possible product finishes	character: size 15
ProductStandardPrice	Unit Prices	Set of all possible unit prices	monetary: 6 digits
ProductLineID	Product Line IDs	Set of all possible product line IDs	integer: 3 digits
OrderedQuantity	Quantities	Set of all possible ordered quantities	integer: 3 digits

Entity Integrity

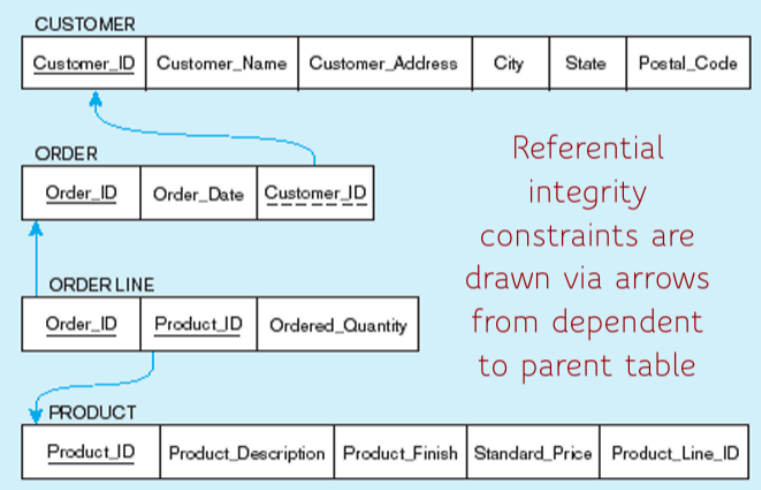
- The entity integrity rule is designed to ensure that every relation has a primary key and that the data values for that primary key are all valid
- A rule that states that **no primary key attribute (or component of a primary key attribute) may be null** → primary key adalah pengidentifikasi utama sehingga TIDAK BOLEH NULL.



Referential Integrity

- Rule that maintains consistency among the rows of two relations.
- Any foreign key value MUST watch a primary key value in the referenced relation (or the foreign key can be null)
  For example: delete rules
  - **Restrict** - don't allow delete of parent side if related rows exist in dependent side → use case: for important/critical data

- **Cascade** - automatically delete dependent side rows that correspond with the parent side row to be deleted → use case: update, for data not as important
- **Set-to-NULL** - set the foreign key in the dependent side to null if deleting from the parent side



Formal Relational Query Language

Query Languages

1. Functional/Procedural (how?)
    a. relational algebra
2. Non-procedural/declarative (what?)
    a. tuple relational calculus
    b. domain relational calculus

Relational Algebra

- Basic operators:
    - select: σ
    - project: Π
    - union: $\cup$
    - set difference: $-$
    - cartesian product: $\times$
    - rename: ρ
- Additional operators:
    - intersection
    - natural join

    - assignment
    - outer join
    - division
- Extended operators:
    - generalized projection
    - aggregation

The operators take one or two relations as inputs and produce a new relation as a result.

Basic Operators

Formal definition:
A basic expression in the relational algebra consists of either one of the following: a relation in the database or a constant relation

Select Operation

Notation:

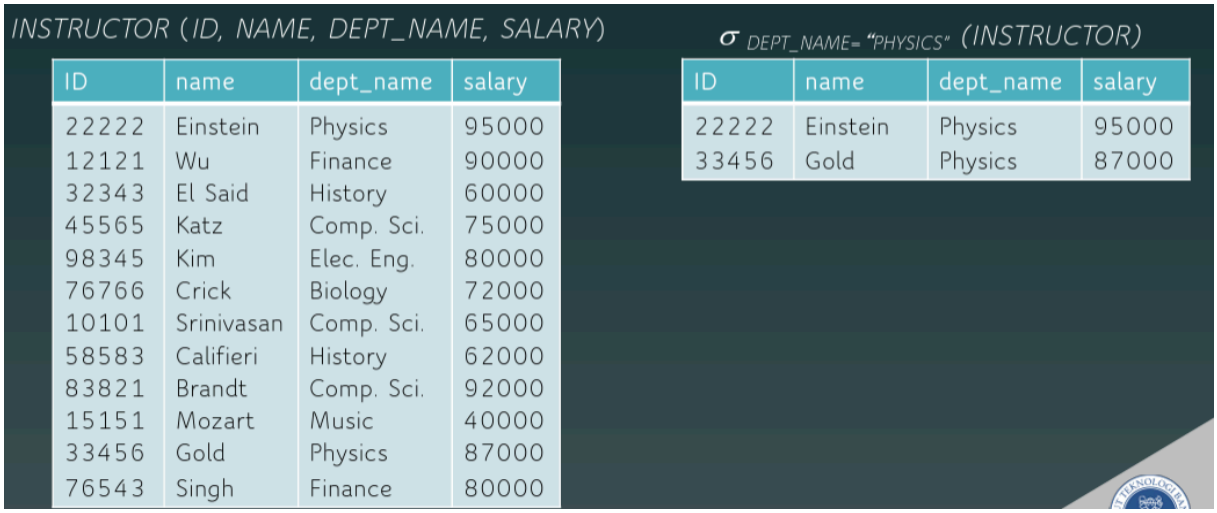
$$\sigma_p(r)$$; p = selection predicate

Defined as:

$$\sigma_p(r) = \{t \mid t \in r \; and \; p(t)\}$$

- where p is a formula in propositional calculus: terms connected by $\wedge \; (and), \; \vee \; (or), \; \neg \; (not)$
- Term = <attribute> *op* <attribute> OR <constant>

Example:



ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

ID	name	dept_name	salary
22222	Einstein	Physics	95000
33456	Gold	Physics	87000

Project Operation

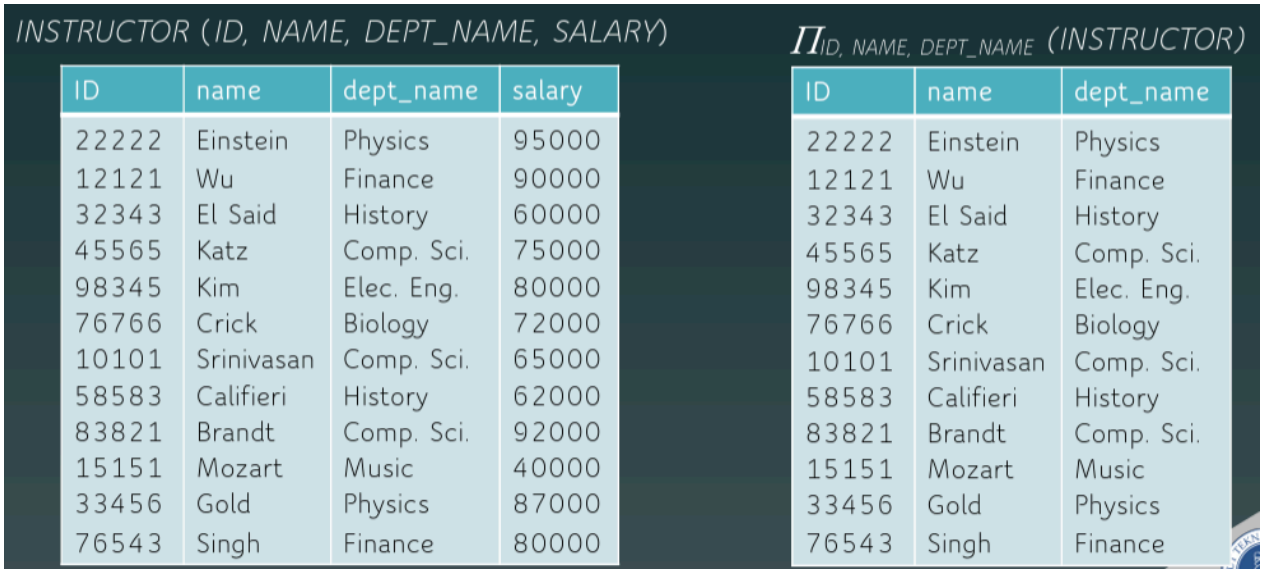
Notation:

$$\Pi_{A1, A2, \ldots, Ak}(r)$$; where A_1, A_2, ... are attribute names and r is a relation name

Defined as:

- The relation of k columns obtained by erasing the columns that are not listed
- Duplicate rows removed from result, since relations are sets

Example:



Composition of Relational Operations

The result of a relational-algebra operation is a relation and therefore some of relational-algebra operations can be composed together into a relational-algebra expression. Consider the query: Find the names of all instructors in the Physics department.

$$\Pi_{name}(\sigma_{deptname="Physics"}(instructor))$$

Union Operation

Notation:

$$r \cup s$$

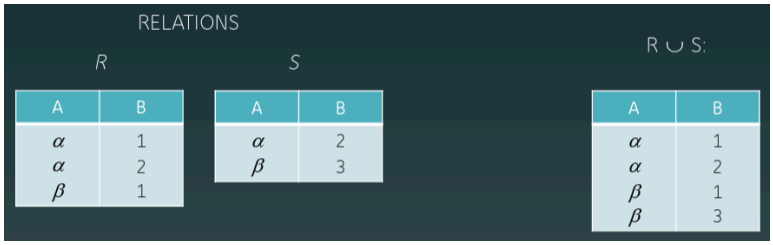
Defined as:

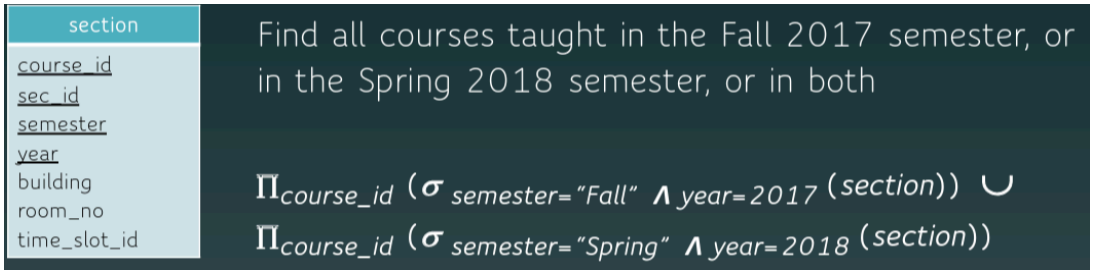
$$r \cup s = \{t \mid t \in r \text{ or } t \in s\}$$

Requirements:

- r, s must have the same **arity**
- the attribute domains must be **compatible**

Example:





Set Difference Operation

Notation:

$$r - s$$

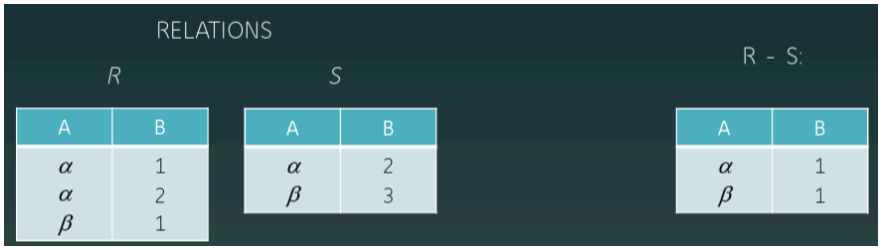
Defined as:

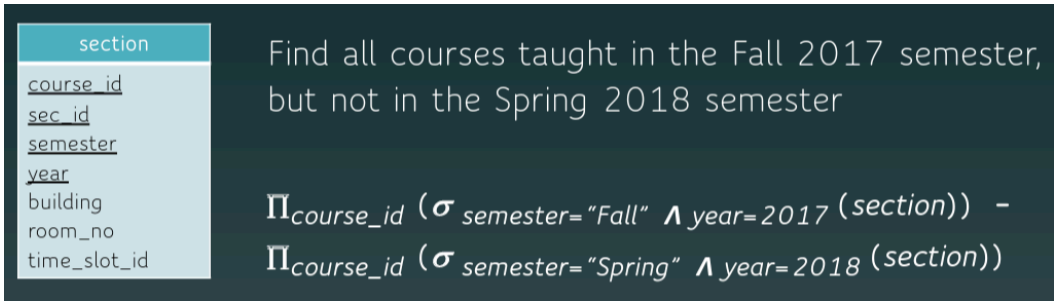
$$r - s = \{t \mid t \in r \text{ and } t \notin s\}$$

Requirements:

Set differences must be taken between compatible relations

Example:





Cartesian-Product Operation

Notation:

$$r \times s$$

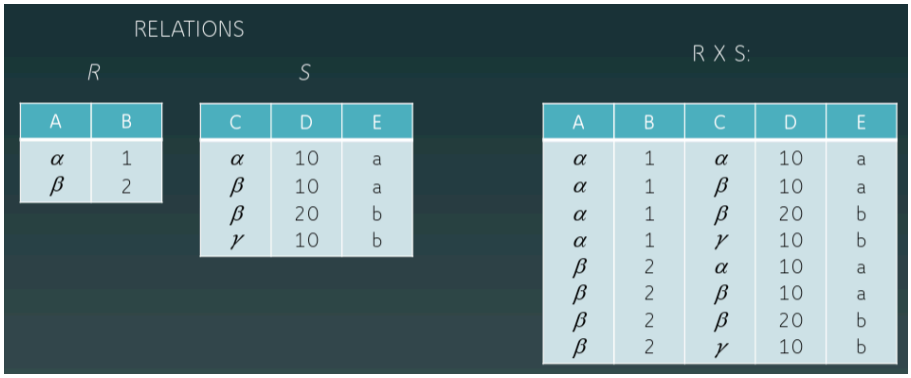
Defined as:

$$r \times s = \{t\, q \mid t \in r \text{ and } q \in s\}$$

Requirements:

- attributes of r(R) and s(S) are disjoint
- if attributes of r(R) and s(S) are not disjoint, then rename OR attach its source relation

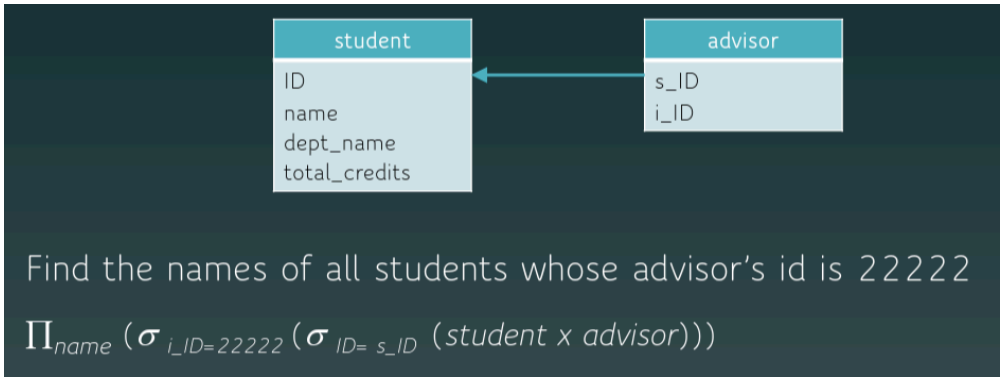
Example:



A	B
α	1
β	2

C	D	E
α	10	a
β	10	a
β	20	b
γ	10	b

A	B	C	D	E
α	1	α	10	a
α	1	β	10	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b



Rename Operation

Notation (relation):

$$\rho_X(E)$$

returns the result of expression E under the name X

Notation (attributes):

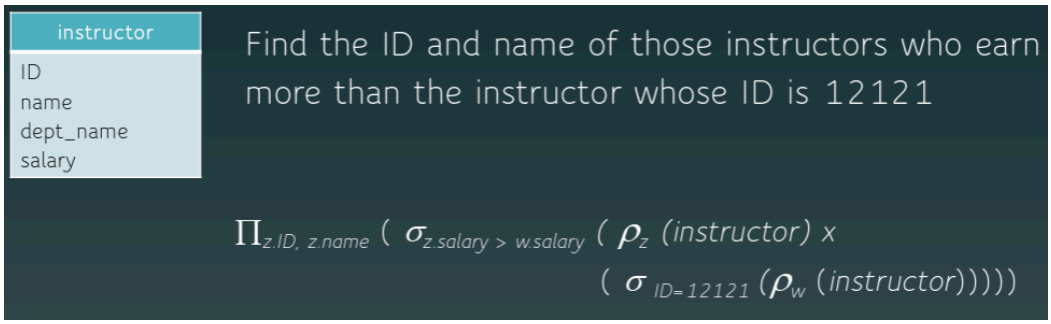
$$\rho_{X(A1,\ A2,\ \ldots,\ An)}(E)$$

returns the result of expression E under the name X, and with the attributes renamed to A1, A2, ..., An

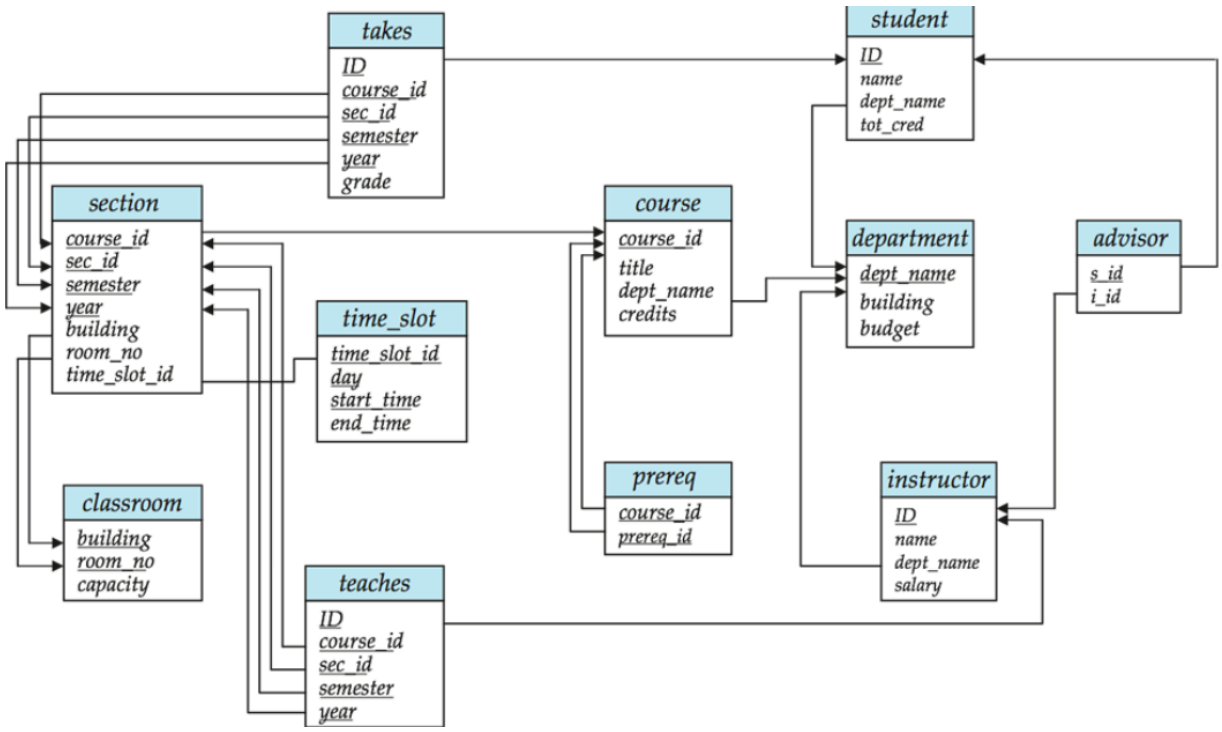
Usage:

- to name the results of relational-algebra expressions
- to refer to a relation by more than one name

Example:



Latihan



Daftar semua mata kuliah (ID, judul, dan sks) di departemen “Comp. Sci.”.

$$\Pi_{courseid,\, title,\, credits}(\sigma_{deptname\, =\, "Comp.\ Sci."}(course))$$

ID, nama, dan departemen asal mahasiswa yang pernah mengambil mata kuliah yang bukan berasal dari departemennya, berikut informasi nama, jumlah sks, dan departemen penyelenggara mata kuliah tersebut.

$$\rho_S(ID,\ name,\ sdeptname,\ totcred)(student)$$

$$\rho_T(ID,\ courseID,\ secID,\ semester,\ year,\ grade)(takes)$$

$$\rho_C(courseID,\ title,\ cdeptname,\ credits)(course)$$

$$\Pi_{S.ID,\, S,name,\, S.sdeptname,\, C.title,\, C.credits,\, C.cdeptname}(\sigma_{S.ID-T.ID\, \wedge\, T.courseID\, =\, C.\, courseID\, \wedge\, S.sdeptname\, \neq C.cdeptname}(S\ \times\ T\ \times\ C))$$

Nama instruktur yang tidak mengajar pada semester 2 tahun 2025.

$$\rho_I(ID,\ name,\ dept,\ salary)(instructor)$$

$$\rho_T(ID,\ courseID,\ secID,\ semester,\ year)(teaches)$$

$$\Pi_{name}(I)\ -\ \Pi_{I.name}(\sigma_{I.ID=T.ID\, \wedge\, semester=2\, \wedge\, year=2025}(I\ \times\ T))$$

ID dan nama semua orang yang ada di university.

$$\Pi_{ID,\, name}(student\ \cup\ instructor)$$

ID instruktur yang berlokasi di gedung “LabTek 5" dan mengajar kelas yang diselenggarakan di gedung “LabTek 8” pada semester 2 tahun 2025.

$\rho_I(ID,\ name,\ dept,\ salary,\ ibuilding)(instructor)$

$\rho_T(ID,\ courseID,\ secID,\ semester,\ year)(teaches)$

$\rho_S(courseID,\ secID,\ semester,\ year,\ sbuilding,\ room,\ slot)(section)$

$\Pi_{I.ID}(\sigma_{I.ID=T.ID \wedge T.courseID=S.courseID \wedge T.secID=S.secID \wedge T.semester=S.semester \wedge T.year=S.year \wedge ibuilding="LabTek\ 5" \wedge sbuilding="LabTek\ 8" \wedge T.semester=2 \wedge T.year=2025}(I \times T \times S))$

Additional Operators

Set intersection

Notation:

$$r \cap s$$

Defined as:

$$r \cap s = \{t \mid t \in r \text{ and } t \in s\}$$

Requirement:

Set intersection must be taken between compatible relations

Example:

Natural join

→ associative & commutative (urutan doesnt matter)

→ akan selalu menyamakan semua atribut yang namanya sama

Notations:

$$r \bowtie s$$

Defined as:

- Let r and s be relations on schema R and S respectively. Then, $r \bowtie s$ is a relation on schema $R \cup S$ obtained as follows:
  If t_r and t_s have the same value on each of the attributes in $R \cap S$, add a tuple t to the result (concatenation of t_r and t_s)

Example:

RELATIONS

R

A	B	C	D
α	1	α	a
β	2	γ	a
γ	4	β	b
α	1	γ	a
δ	2	β	b

S

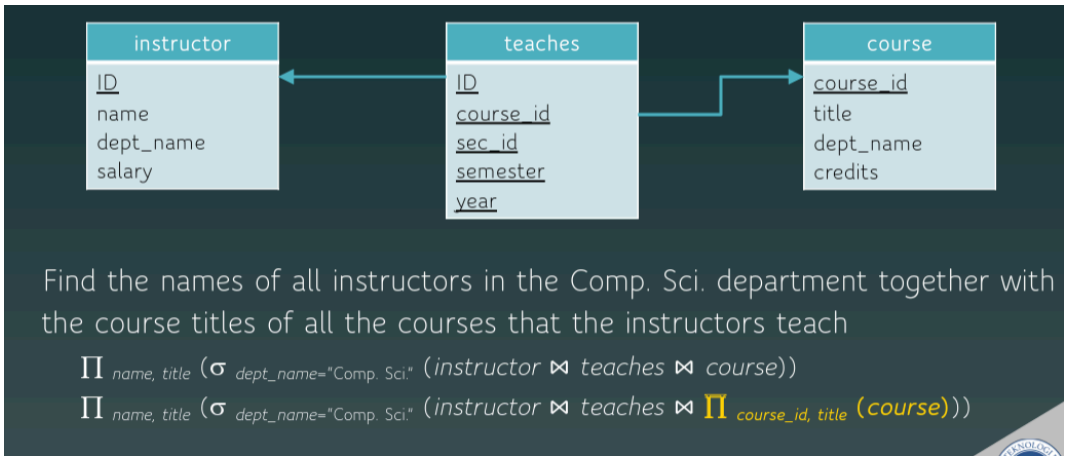
B	D	E
1	a	α
3	a	β
1	a	γ
2	b	δ
3	b	ε

R ⋈ S

A	B	C	D	E
α	1	α	a	α
α	1	α	a	γ
α	1	γ	a	α
α	1	γ	a	γ
δ	2	β	b	δ

Note: natural join can be expressed using basic operations.
e.g. $r \bowtie s$ can be written as

$$\Pi_{r.A,\ r.B,\ r.C,\ r.D,\ s.E}\ (\sigma_{r.B\ =\ s.B\ \wedge\ r.D\ =\ s.D}\ (r\ \times\ s))$$



Find the names of all instructors in the Comp. Sci. department together with the course titles of all the courses that the instructors teach

$$\Pi_{name,\ title}\ (\sigma_{dept\_name="Comp.\ Sci."}\ (instructor \bowtie teaches \bowtie course))$$

$$\Pi_{name,\ title}\ (\sigma_{dept\_name="Comp.\ Sci."}\ (instructor \bowtie teaches \bowtie \Pi_{course\_id,\ title}\ (course)))$$

Theta Join

$$r \bowtie_{\theta} s\ =\ \sigma_{\theta}(r \times s)$$

Assignment

The assignment operation (←) provides a convenient way to express complex queries

- Write query as a sequential program consisting of:
  - a series of assignments
  - followed by an expression whose value is displayed as a result of the query
- Assignment must always be made to a temporary relation variable

Example:

Example: Find all instructor in the "Physics" and "Music" department.

$$Physics \leftarrow \sigma_{dept\_name="Physics"}(instructor)$$

$$Music \leftarrow \sigma_{dept\_name="Music"}(instructor)$$

$$Physics\ \cup\ Music$$

Outer join

Notation:

- Left outer join: ⟕
- Right outer join: ⟖
- Full outer join: ⟗

Defined as:

Computes the join then adds tuples from one relation that does not match tuples in the other relation to the result of the join

Express outer join (e.g. r ⟗ s) using basic [and join - to simplify] operations

$$(r \bowtie s) \cup ((r - \prod_R (r \bowtie s)) \times \{(null, \ldots, null)\})$$

Uses null values

- null signifies that the value is unknown or does not exist
- all comparisons involving null are (roughly speaking) false by definition

Null values:

Division

Notation:

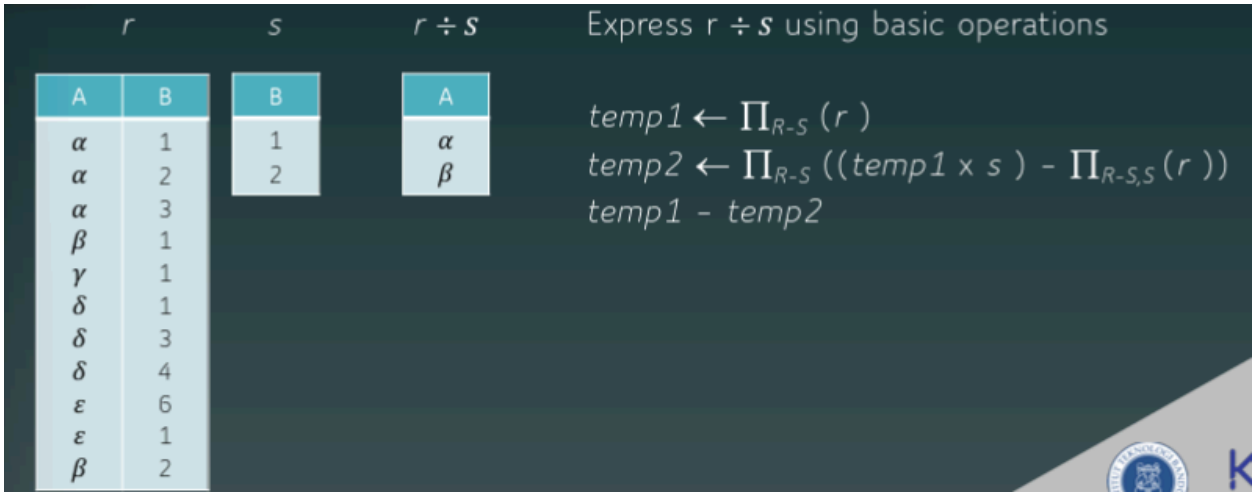
$r \div s \rightarrow$ hasilnya: atribut r yang ada di s

Defined as:

Given relations r(R) and s(S), such that $S \subset R$, $r \div s$ is the largest relation t(R-S) such that $t \times s \subseteq r$

Example:

let $r(ID, course\_id) = \prod_{ID,\ course\_id}(takes)$ and
$s(course\_id) = \prod_{course\_id}(\sigma_{dept\_name="Biology"}(course))$
then r ÷ s = students who have taken all courses in the Biology Dept.



Extended Operators